

PCP

LINUX

Prepared by

Prof. Chauhan Bhavik | Assistant professor | MScIT

- 308-A, SSASIT
- Communication: bhavik.chauhan@ssasit.ac.in
- Linkedin: <https://www.linkedin.com/in/bhavik-chauhan-4972a8aa/>
- Github: <https://github.com/ChauhanBhavik5>

Content

- ☐ History & Features of Linux,
- ☐ Linux Architecture,
- ☐ File System of Linux,
- ☐ Hardware Requirements of Linux,
- ☐ Various flavors of Linux,
- ☐ Linux Standard Directories,
- ☐ Functions of Profile and Login Files in Linux,
- ☐ Linux Kernel,
- ☐ Installation of Linux,
- ☐ Introduction of Ubuntu, Fedorra, RedHat Linux

History of Linux

- Unix (1969): Developed at Bell Labs (Ken Thompson & Dennis Ritchie).
- Problems with Unix: Expensive, closed-source, limited to big companies.
- Minix (1987): Small Unix-like OS for education (Andrew Tanenbaum).
- Linux (1991): Linus Torvalds created a free, open-source kernel.
- Open Source Movement: GNU Project + Linux kernel = GNU/Linux.
- Today: Used in servers, cloud, mobile (Android), supercomputers.

Timeline: Unix (1969) → Minix (1987) → Linux (1991) → Modern Linux Distros.

Features of Linux

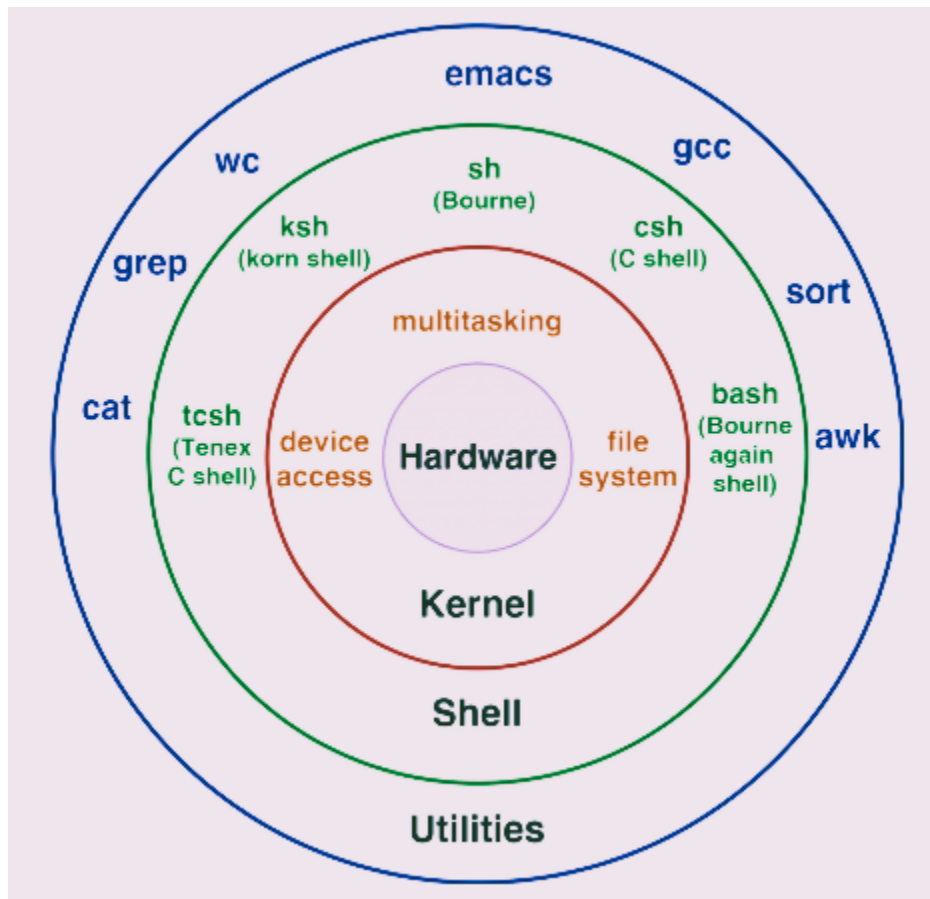
- **Multi-user: Many users can log in simultaneously.**
 - More than one person can log into the same system at once.
 - Each user has their own files, settings, and permissions.
 - Users cannot easily interfere with each other's data.
 - Example: A server can host websites for thousands of users at the same time.
- **Multitasking: Run multiple processes at the same time.**
 - Linux can run many programs/processes at the same time.
 - Background jobs (like downloads) keep running while you work.
 - System resources (CPU, memory) are shared efficiently.
 - Example: You can browse the web, edit a file, and play music together.
- **Portability: Runs on PCs, servers, mobiles, IoT devices.**
 - Linux can run on almost any hardware: PC, laptop, mobile, supercomputer, IoT device.
 - The same software can be compiled and used across different platforms.
 - Doesn't require very high-end hardware (can run on old PCs too).
 - Example: Android (phones) and Ubuntu (PCs) are both Linux-based.
- **Security: File permissions, user groups, firewalls.**
 - Strong file permissions (read, write, execute) control access.
 - User accounts & groups provide isolation.

- Built-in security tools like firewalls (iptables, ufw).
- Much less targeted by viruses compared to Windows.
- **Open Source: Source code is freely available.**
 - Source code of Linux is freely available to everyone.
 - Anyone can study, modify, or improve it.
 - A large global community maintains and supports it.
 - Free to use → no licensing fees.
- **Stability & Performance: Uptime of years, widely used in servers.**
 - Linux can run for months or years without crashing.
 - Very efficient at handling heavy loads (databases, servers, cloud).
 - Used in 90%+ of supercomputers and most web servers.
 - Example: Companies like Google, Facebook, Amazon run on Linux servers.

Question: “Why do you think most web servers use Linux instead of Windows?”

Linux Architecture

Layers of Linux OS:



Hardware (Base Layer)

- Includes CPU, memory (RAM), hard disk, and I/O devices (keyboard, mouse, network card).
- Provides the raw resources needed by the operating system.
- Hardware does not interact with applications directly → it needs the kernel.
- Example: Your browser doesn't talk to RAM directly; the kernel manages it.

Kernel (Kernel Space):

- The core of Linux – acts as a bridge between hardware and user programs.
- Key Points:
 - Resource Manager: Manages CPU scheduling, memory allocation, file system, devices.
 - Monolithic Kernel:
 - All essential services (memory, file system, device drivers) run inside kernel mode.
 - Makes Linux powerful and fast.
 - System Calls: Provides an interface for programs to request kernel services.
 - Device Drivers: Enable hardware (keyboard, USB, printer) to work with Linux.
 - Example: When you open a file, your text editor uses system calls → kernel → hard disk.

User Space

- This is where users interact with Linux.

Components:

- Shell:
 - Command-line interpreter (bash, zsh, fish).
 - Converts user commands into kernel-understandable instructions.
 - Example: When you type ls, the shell sends a request to the kernel.
- Libraries:
 - Pre-written code that applications reuse (e.g., glibc).
 - Provides standard functions (open file, print output).
- Applications:
 - Programs for end users (text editors, browsers, servers).
 - Built on top of libraries + shell.

Linux vs Windows vs Unix

Feature	Linux	Windows	Unix
Cost	Free & Open Source	Licensed (paid)	Expensive
Source Code	Open	Closed	Closed
Users	Multi-user	Multi-user (limited)	Multi-user
Security	Very strong	Vulnerable (more malware)	Strong
Portability	Runs everywhere	Limited	Limited
Usage	Servers, mobile, cloud, desktops	Desktops, gaming	Legacy servers
Example	Fedora, Red Hat, Debian, Android	Windows 10, 11 (desktops & laptops), Windows Server	AIX (IBM), HP-UX (HP), Solaris (Oracle)

Linux File System & Directories

Linux File System

Hierarchical Structure

- Linux uses a single-rooted tree structure.
- Everything starts at the root directory /.
- Directories branch out like a tree (like C:\ in Windows).
- Devices, files, and processes are all represented as files in Linux.

Example:

```
/
|
├── home/
|   └── user/
├── etc/
├── var/
└── bin/
```

Inode Concept

- Each file/directory has an **inode** (index node).
- Inode stores **metadata** (permissions, owner, size, timestamps, location on disk).
- Inode does **not store file name**, just file info.
- File name is mapped to inode via directory entry.
- Example command:

◆ `ls -li file.txt` # shows inode number

Linux Standard Directories

Directory	Purpose
/	Root directory (starting point of everything)
/home	User home directories (/home/student)
/etc	System configuration files (/etc/passwd, /etc/hosts)
/var	Variable data like logs (/var/log), spool, mail
/bin	Essential user commands (ls, cp, mv)
/usr	User programs, libraries, and documentation
/tmp	Temporary files (cleared on reboot)
/dev	Device files (e.g., /dev/sda for hard disk, /dev/tty for terminal)
/proc	Virtual file system showing kernel & process info (/proc/cpuinfo, /proc/meminfo)

File Permissions Basics

- Each file has 3 types of permissions:
 - Read (r), Write (w), Execute (x)
- Assigned to 3 categories of users:
 - Owner, Group, Others
- Command
 - ls -l
- Example output:
 - -rwxr-xr-- 1 user group 1024 Sep 20 [file.sh](#)

Breakdown:

- rwx → Owner can read, write, execute
- r-x → Group can read, execute
- r-- → Others can only read

Hardware Requirements of Linux

Hardware Requirements

Different Linux distributions (distros) have different hardware requirements. Here's a basic guide:

Component	Minimum Requirements	Recommended for Smooth Performance
CPU	1 GHz Processor	2 GHz Dual-Core Processor or higher
RAM	512 MB to 1 GB	2 GB or more
Storage	5 GB of disk space minimum	20 GB or more
Graphics	Any basic graphics support	Higher-end graphics (optional for gaming / graphics - heavy tasks)
Network	Network interface for internet access	Ethernet or Wi-Fi

Installation Process of Linux

Types of Installation

➤ **Dual Boot:**

- Install Linux alongside Windows or macOS.
- You can choose the operating system when booting the computer.
- Ideal for users who want to keep their old OS but also want Linux for development or testing.

➤ **Virtual Machine (VM):**

- Install Linux inside a VM (e.g., VirtualBox, VMware) while keeping your current OS.
- No need to modify your hard drive or partition.
- Ideal for learning and testing without making permanent changes.

➤ **Windows Subsystem for Linux (WSL):**

- Allows you to run a Linux distribution within Windows (Windows 10+).
- Provides access to a full Linux environment directly within Windows.

Linux Kernel

Role of the Linux Kernel

- The kernel is the core part of Linux. It manages hardware, system resources, and **enables communication** between hardware and software.
- It's responsible for:
 - Memory management
 - Process management
 - Device management
 - File systems

Monolithic Kernel

Linux uses a monolithic kernel:

- All essential services (memory management, device drivers, system calls) run within kernel space.
- Unlike microkernel (other OS), which runs services outside the kernel, Linux's monolithic kernel is faster and more efficient

Kernel Modules & Device Drivers

- Device Drivers: Handle communication between hardware (e.g., printer, USB device) and the kernel.
- Kernel Modules: These are pieces of code that can be loaded or unloaded into the kernel at runtime without restarting the system.
 - Example: lsmod to see loaded modules, modprobe to load a new module.

Example:

- You can insert/remove a device like a USB drive without restarting Linux.
- Example commands:
 - lsmod: Lists currently loaded kernel modules.
 - modprobe <module_name>: Loads a module.
 - rmmod <module_name>: Removes a module.

What is a Linux Distribution?

Definition:

- A Linux Distribution (Distro) = Linux Kernel + System Utilities + Software Packages + Package Manager + Desktop Environment
- Each distro customizes Linux for specific needs — servers, desktops, security testing, or development.
- Key Points:
 - All distros use the same Linux kernel, but differ in package management, user interface, and purpose.
 - Some are designed for beginners (Ubuntu), others for enterprises (RedHat), or security professionals (Kali)

Various Flavors of Linux

Distro	Base	Package Manager	Target Users / Use Case	Highlights
Debian	Independent	apt, dpkg	General users, stable servers	Very stable, basis for Ubuntu
Ubuntu	Debian	apt, snap	Beginners, developers	User-friendly, large community
Fedora	RedHat	dnf, rpm	Developers, testers	Latest technologies, short release cycle
RedHat Enterprise Linux (RHEL)	Fedora	yum, dnf, rpm	Enterprises, servers	Commercial support, certified OS

CentOS / AlmaLinux / Rocky Linux	RHEL	yum, dnf, rpm	Servers, sysadmins	RHEL-compatible, free alternative
Arch Linux	Independent	pacman	Advanced users	Rolling release, full customization
SUSE / openSUSE	Independent	zypper, rpm	Enterprises, developers	YaST configuration tool
Kali Linux	Debian	apt	Cybersecurity professionals	Preloaded with penetration testing tools

Introduction to Key Distributions

Ubuntu

- Base: Debian
- Target Users: Students, developers, beginners
- Package Manager: apt (Advanced Package Tool)
- Features:
 - User-friendly interface (GNOME Desktop)
 - LTS (Long Term Support) versions every 2 years
 - Huge community and online support
 - Easy software installation with Software Center
 - Cloud-ready (used in AWS, Azure)
- Example Commands:
 - ☐ `sudo apt update`
 - ☐ `sudo apt install vim`

Fedora

- **Base:** RedHat (Community version)
- **Target Users:** Developers, testers
- **Package Manager:** **dnf** (Dandified Yum)
- **Features:**
 - Cutting-edge updates and technologies
 - Short release cycle (every ~6 months)
 - Default GNOME desktop
 - Often used to test features before adding them to RHEL
- **Example Commands:**
 - ☐ `sudo dnf update`
 - ☐ `sudo dnf info firefox`

RedHat Enterprise Linux (RHEL)

- **Base:** Fedora
- **Target Users:** Enterprises, system administrators
- **Package Manager:** **yum** / **dnf** with **.rpm** packages
- **Features:**
 - Commercial support from RedHat
 - Enterprise stability and security
 - Certified for servers and cloud deployments
 - Requires subscription for updates

Comparison Chart

Feature	Ubuntu	Fedora	RedHat
Base	Debian	Fedora (Community)	Fedora
Type	Free & Open Source	Free & Open Source	Commercial (Enterprise)
Package Manager	<code>apt</code>	<code>dnf</code>	<code>yum / dnf</code>
Target Users	Beginners, Students	Developers, Testers	Enterprises
Release Cycle	LTS (2 years)	Rapid (6 months)	Long-term & Stable
Support	Community	Community	Paid (Enterprise)
Example Command	<code>sudo apt update</code>	<code>sudo dnf info nano</code>	<code>sudo yum install httpd</code>

Notes:

- All Linux distros share the same kernel, but differ in tools, UI, and package management.
- Ubuntu - best for **learning** and **beginners**.
- Fedora - latest features for **developers**.
- RedHat - **enterprise**-grade stability and support.

Login vs Non-login Shell

Shell

- The **shell** is a command-line interpreter that allows users to interact with the Linux system.
- Common shells: **bash**, **zsh**, **sh**, **fish**

Login Shell:

- When a user logs in through **console**, **SSH**, or **terminal login**, the shell is started as a **login shell**.
- It reads **startup files** like `/etc/profile`, `~/.bash_profile`, or `~/.profile`.

Example:

☐ `ssh user@server` # Opens a login shell

Non-login Shell:

- When a terminal is opened inside a GUI or when a new bash is spawned manually (bash), it's a non-login shell.
- Reads `~/.bashrc` instead of profile files.

Example:

☐ `bash` # Opens a non-login shell

Key Difference Table:

Type	Triggered When	Files Read
Login Shell	User logs in (console/SSH)	<code>/etc/profile</code> , <code>~/.bash_profile</code> , <code>~/.profile</code>
Non-login Shell	Opening a terminal or running <code>bash</code> inside	<code>~/.bashrc</code>

Profile and Login Files

File	Location	Purpose
<code>/etc/profile</code>	System-wide	Executed for all users at login. Sets PATH, environment variables globally.
<code>~/.profile</code>	User's home directory	Personal startup file for login shell. User-specific environment setup.
<code>~/.bash_profile</code>	User's home directory	Used by bash as a login shell; can call <code>.bashrc</code> inside it.
<code>~/.bashrc</code>	User's home directory	Executed for every new terminal (non-login). Contains aliases and shell customizations.

Common Environment Variables:

Variable	Purpose	Example
PATH	List of directories searched for commands	<code>/usr/bin:/bin:/usr/local/bin</code>
HOME	User's home directory	<code>/home/student</code>
USER	Logged-in username	<code>student</code>
SHELL	Default shell	<code>/bin/bash</code>
PWD	Current directory	<code>/home/student/Desktop</code>

ALL THE BEST